

ТЕХНИЧЕСКОЕ ЗАДАНИЕ
в рамках практики в IVI для проекта на тему
«ЭТО МОЯ ПЕРЕГОВОРКА!»

Оглавление

Описание идеи.....	3
Общие требования к проекту.....	5
Требования к реализации проекта	7
Требования к определенным пунктам.....	11

Описание идеи

Название проекта: «Booked!»

Проблема, которую решает сервис: сотрудникам часто приходится тратить время на поиск свободной переговорной, а отсутствие единого расписания приводит к конфликтам при бронировании. Сервис помогает быстро найти подходящее пространство и избежать затруднительных ситуаций.

Целевая аудитория: сотрудники и руководители команд, регулярно организующие встречи и нуждающиеся в быстром поиске и бронировании свободных переговорных комнат.

Основная ценность: сервис экономит время сотрудников, автоматически подбирая подходящие по разным параметрам переговорные.

Ключевая бизнес-логика:

1. Пользователь указывает параметры встречи:
 - дату;
 - время начала;
 - продолжительность;
 - количество участников;
 - при необходимости выбирает дополнительное оборудование.
2. Программа анализирует все переговорные комнаты и предлагает те, которые:
 - свободны в указанный промежуток времени;
 - могут вместить необходимое количество сотрудников;
 - соответствуют выбранным требованиям по оборудованию.
3. Пользователь выбирает одну из предложенных переговорных и создает бронирование;
4. Перед сохранением система проверяет пересечения с уже существующими бронированиями выбранной переговорной. При наличии конфликта заявка не создается;

5. При успешном бронировании информация отображается в расписании переговорной. Выбранный временной интервал становится недоступным для пользователей;
6. Для каждой переговорной создается детальная страница, где есть расписание в виде таймлайна с занятыми и свободными временными интервалами;
7. Гость (неавторизованный пользователь) может только просматривать расписание переговорных и их страницы, выполнять поиск нужной переговорной (см. п. 1);
8. Пользователь (авторизованный пользователь) может то же самое, что и гость, и может изменять (создавать/редактировать/удалять) **только свои** заявки. При каждом повторном изменении выполняется проверка на пересечения с уже существующими бронированиями (см. п. 4);
9. Администратор (суперпользователь) может то же самое, что и пользователь и может просматривать и редактировать любые бронирования и управлять списком переговорных комнат: их вместимостью и доступным оборудованием;
10. Гость не имеет доступа к созданию, редактированию, удалению заявок. В случае попытки получить доступ, система перенаправляет гостя на страницу логина.
11. При отсутствии полностью подходящих переговорных система сообщает об этом и предлагает изменить параметры поиска;
12. При наличии нескольких подходящих переговорных система сортирует их по минимально достаточной вместимости.

Общие требования к проекту

1. Python и Django

- Python 3.10+;
- Django 4.2+.

2. Базы данных

- PostgreSQL.

3. Модели

- 4+ модели;
- минимум 2 типа связей между моделями (ForeignKey, ManyToMany, OneToOne);
- кастомная модель пользователя;
- 1+ кастомный метод или property у модели.

4. Views

- 2+ Class-based View (ListView, CreateView, UpdateView, DetailView);
- полный на основной сущности.

5. Реализация бизнес-логики:

- реализовать полный CRUD для сущности «Запись»;
- реализовать отображение списка комнат с пагинацией (5-10 на страницу);
- реализовать поиск переговорных комнат по одному полю (названию/вместимости/оборудованию);
- реализовать регистрацию, авторизацию, выход пользователя;
- администратор имеет полный доступ ко всем объектам системы;
- реализовать административную панель с возможностью управления переговорными комнатами и бронированиями.

6. Тесты

- Pytest или unittest;
- покрытие кода тестами – не менее 70%;
- все тесты проходят.

7. Интерфейс

- 5+ страницы;
- наследование шаблонов (base.html с navbar и футером);
- CSS-фреймворк;
- отображение ошибок форм;
- навигация работает (ссылки ведут куда надо).

8. Git

- GitHub;
- 10+ коммитов на человека;
- осмысленные сообщения;
- .gitignore для Python/Django.

9. Архитектурные артефакты (в репозитории)

- схема базы данных;
- схема сервисов (слои приложения);
- схема взаимодействия (1 сценарий);
- декомпозиция (задачи, часы, ответственные).

Требования к реализации проекта

Функциональные требования:

Роли пользователей:

1. Гость (неавторизованный пользователь)

- просмотр списка переговорных комнат;
- просмотр информации о переговорной комнате и ее расписания;
- поиск переговорных по одному полю (названию/вместимости/оборудованию).

2. Пользователь (авторизованный пользователь)

Имеет все возможности гостя, а также:

- создание бронирований переговорной комнаты;
- просмотр списка собственных бронирований;
- редактирование/удаление собственного бронирования;
- поиск свободных переговорных по параметрам встречи;
- просмотр доступных временных интервалов для выбранной переговорной.

3. Администратор (суперпользователь)

Имеет все возможности пользователя, а также:

- создание переговорных комнат;
- редактирование информации о переговорных комнатах;
- удаление переговорных комнат;
- ограничение доступа к переговорным комнатам;
- просмотр, редактирование, удаление любых бронирований.

Основные сценарии использования:

1. Регистрация и авторизация

Пользователь может:

- зарегистрироваться;
- войти в систему;
- выйти из системы.

2. Просмотр переговорных

Пользователь может:

- посмотреть список переговорных комнат;
- открыть карточку выбранной переговорной;
- посмотреть вместимость комнаты;
- посмотреть доступное оборудование;
- ознакомиться с расписанием переговорной на выбранную дату.

3. Поиск переговорной

Пользователь указывает:

- дату встречи;
- время начала;
- продолжительность;
- количество участников;
- необходимое оборудование.

4. Создание бронирования

Пользователь выбирает переговорную комнату и подтверждает бронирование.

Система проверяет:

- отсутствие пересечения с другими бронированиями;
- соответствие вместимости количеству участников;
- доступность переговорной.

5. Редактирование бронирования

Пользователь может изменить:

- переговорную;
- дату;
- время;
- продолжительность;
- количество участников;

6. Удаление бронирования

Пользователь может:

- полностью удалить бронирование.

7. Управление переговорными

Администратор может:

- создавать новые переговорные;
- изменять характеристики переговорных;
- удалять переговорные;
- ограничивать доступ к переговорным.

В самом конце работы необходимо записать короткий ролик с демонстрацией работы проекта.

Требования к видео:

- **Длительность:** не более 7 минут;
- **Содержание:** показать работу проекта: как реализовывались ключевые фичи, что было самым сложным, и выводы проделанной работы;
- **Формат:** ссылку на видео добавить в README репозитория.

Что должно быть в репозитории

README.md	Подробнее в «Требования к документации»
ER_diagram.drawio	Схема БД
DECOMPOSITION.md	Таблица с задачами, часами, ответственными
requirements.txt	Все зависимости
.env.example	Пример переменных окружения
manage.py + проект	Рабочий код
tests/	Папка с тестами (покрытие > 70%)

	Настройка PostgreSQL	1	Милютин
	Настройка Git и структуры проекта	0.5	Милютин
	Подключение Bootstrap	0.5	Ханана
Проектирование	Проектирование моделей и связей	2	Ахмедзянов
	Создание ER-диаграммы	1.5	Ахмедзянов
Модели и БД	Кастомная модель User	1	Милютин
	Модель MeetingRoom	1	Милютин
	Модель Booking	2	Понарин
	Модель Equipment	1	Ханана
	Создание миграций	0.5	Шулаев
	Наполнение тестовыми данными	1	Шулаев
Авторизация	Регистрация пользователя	2	Ахмедзянов
	Авторизация и выход	1	
	Настройка прав доступа	1.5	
Переговорные	Просмотр списка переговорных	1.5	Шулаев
	Детальная страница переговорной	1	
	CRUD переговорных (админ)	4	
Бронирования	Создание бронирования	3	Понарин
	Редактирование	2	
	Удаление	1	

	Просмотр своих бронирований	1	
Бизнес-логика	Проверка пересечений бронирования	3	Ханана
	Проверка вместимости комнаты	1	Ханана
	Подбор переговорных по параметрам	3	Ханана
	Фильтрация по оборудованию	2	Ахмедзянов
	Формирование временных слотов	3	Милютин
Расписание	Отображение таймлайна переговорной	4	Милютин
Поиск	Поиск по одному параметру	2	Милютин
Админ-панель	Настройка отображения моделей, фильтров, поиска	2	Милютин
Интерфейс	Создание базового шаблона	2	Ханана
	Страницы регистрации и выхода	1.5	Ханана
	Страница списка переговорных	1.5	Понарин
	Страница переговорной	1.5	Шулаев
	Формы создания и редактирования	2	Ахмедзянов
	Страница поиска	2	Ханана
	Страница своих бронирований	1.5	Понарин
Тестирование	Тесты моделей	2	Понарин
	Тесты представлений	3	Понарин
	Тесты форм	2	Шулаев

	Тесты бизнес-логики	3	Ахмедзянов
Документация	README	2	Ханана
	Подготовка .env.example	0.5	Понарин
	Подготовка скриншотов	1	Понарин
	Подготовка видео	3	Понарин
Итог		79	

Требования к административной панели:

В административной панели должны быть зарегистрированы все модели системы.

Для каждой модели должны быть настроены:

- отображение основных полей (list_display);
- поиск (search_fields) минимум для одной модели;
- фильтрация (list_filter) минимум для одной модели.

Требования к пользовательскому интерфейсу:

В приложении должны присутствовать следующие страницы:

- главная страница (список переговорных);
- страница переговорной комнаты;
- страница поиска переговорных;
- страница создания бронирования;
- страница редактирования бронирования;
- страница «Мои бронирования»;
- страница входа;
- страница регистрации.

Интерфейс должен:

- использовать Bootstrap 5;

- иметь единый базовый шаблон (base.html);
- корректно отображать сообщения об ошибках форм;
- обеспечивать удобную навигацию между страницами.

Тесты должны обязательно покрыть:

- Создание модели (проверить, что сохраняется в БД);
- `__str__` метода модели;
- Валидацию формы (неправильные данные ведут к ошибке);
- Доступ к странице списка (статус 200);
- Доступ к детальной странице (статус 200);
- Доступ к созданию без логина (редирект на логин);
- Доступ к редактированию без логина (редирект на логин);
- Создание объекта через форму (редирект на детальную страницу);
- Редактирование объекта через форму.

Требования к документации:

README.md должен содержать:

- Как развернуть проект (пошагово):
 1. Клонировать репозиторий;
 2. Создать виртуальное окружение;
 3. Установить зависимости (pip install -r requirements.txt);
 4. Настроить .env (пример в .env.example);

`.env.example` (файл в корне):

```
text
SECRET_KEY=your_secret_key
DEBUG=True
DB_NAME=your_db_name
DB_USER=your_db_user
DB_PASSWORD=your_db_password
DB_HOST=localhost
DB_PORT=5432
```

5. Выполнить миграции;

6. Создать суперпользователя;

7. Docker Compose.

- Как запустить тесты;
- Краткое описание моделей (что хранится, как связаны);
- 2-3 скриншота работающего приложения;
- Видео-демо.